



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2233 Programación Avanzada (2026-1)

Tarea 2

Entrega

- Tarea y README.md
 - **Fecha y hora oficial:** domingo 17 de mayo de 2026, 20:00.
 - **Lugar:** Repositorio personal de GitHub — Carpeta: Tareas/T2/.
El código debe estar en la rama (*branch*) por defecto del repositorio: *main*.
 - **Pauta de corrección:** [en este enlace](#).
 - **Bases generales de tareas (descuentos):** [en este enlace](#).
 - **Formulario utilización de IA:** [en este enlace](#). Se cerrará el domingo 17 de mayo, 23:59.
- **Ejecución de tarea:** La tarea será ejecutada **únicamente** desde la terminal del computador. Además, durante el proceso de corrección, se cambiará el nombre de la carpeta “T2/” por otro nombre y se ubicará la terminal dentro de dicha carpeta antes de ejecutar la tarea.

Objetivos

- Entender y aplicar estructuras ligadas para el manejo de datos.
- Entender y aplicar el paradigma de programación funcional para resolver un problema.
- Manejar datos de forma eficiente utilizando herramientas de programación funcional:
 - Uso de generadores, funciones generadoras y envío de valores a función generadora.
 - Uso de estructuras por comprensión.
 - Uso e investigación de las librerías `functools`, `itertools` y `collections`.
- Utilizar conceptos de interfaces y `PyQt6` para implementar una aplicación gráfica e interactiva. Además en esta se debe entender y aplicar los conceptos de *back-end* y *front-end*.
- Aplicar conocimientos de señales.

Índice

1. <i>DCCornucopia</i>	3
2. Flujo del programa	3
3. Archivos	4
4. Programación Funcional	6
4.1. Datos provenientes de archivos	6
4.1.1. <code>Juguete</code>	6
4.1.2. <code>JugueteHabitat</code>	7
4.1.3. <code>HabitatObjeto</code>	7
4.1.4. <code>ObjetoRecurso</code>	8
4.1.5. <code>RecursoRecurso</code>	8
4.1.6. <code>JugueteRecurso</code>	8
4.1.7. <code>JugueteObjeto</code>	9
4.1.8. <code>PeriodoDia</code>	9
4.2. Carga de datos de <code>namedtuples</code>	10
4.3. Datos de ejecución	11
4.3.1. Clase <code>NodoLigado</code>	11
4.3.2. Hábitat	11
4.3.3. <code>JugueteProductor</code>	12
4.3.4. <code>Objeto</code>	12
4.3.5. <code>Recurso</code>	12
4.4. Consultas	13
4.4.1. Consultas que solo usan datos de archivos	13
4.4.2. Consultas que usan datos de ejecución	16
4.4.3. Consulta de simulación	18
5. Interfaz Gráfica e Interacción	19
5.1. Modelación del programa	19
5.2. Ventanas	19
5.2.1. Ventana de Inicio	20
5.2.2. Ventana de Índice de Juguetes	20
5.2.3. Ventana de Simulación de juego	21
6. <i>Tests</i>	23
6.1. Ejecución de <i>tests</i>	23
7. <i>utils.pyc</i>	24
7.1. <code>Namedtuples</code>	24
7.2. <code>NodoLigadoAbstracto</code>	24
8. <code>.gitignore</code>	24
9. Importante: Corrección de la tarea	25
10. Restricciones y alcances	26

1. *DCCornucopia*



El DCC está en remodelación, dado esto ha sido evacuado y los juguetes de las oficinas quedaron abandonados. Un día, estos juguetes cobraron vida, puede que la mezcla del cemento triturado con el aire de pizza y café hayan hecho algo, o algo más que un bit haya cambiado por un rayo solar, sin embargo, ahora tu misión es que estos juguetes puedan sobrevivir hasta que el DCC vuelva a abrir sus puertas.

2. Flujo del programa

En *DCCornucopia*, el principal objetivo será poder realizar consultas en torno a distintas bases de datos que tendrás disponibles. Esta tarea comenzará con funciones para acceder a archivos de distintos tamaños. Tu objetivo en esta sección será almacenar la información de los archivos en `namedtuples` para poder utilizarla más adelante.

Luego, utilizarás tus habilidades de programación funcional y generadores para realizar consultas, las cuales darán información específica sobre las distintas hábitats, juguetes, objetos, recursos o periodos del día. Por último, utilizando los contenidos de interfaces gráficas, deberás crear en total 3 ventanas: una ventana de entrada que da la bienvenida al programa, otra ventana principal para observar los juguetes disponibles, y por último una para simular *DCCornucopia*.

La realización de la tarea se dividirá en dos partes principales:

1. En la [Sección 4: Programación Funcional](#), se pedirá que implementes las funciones mencionadas más adelante. Para que el código sea ordenado, estas funciones ya están declaradas en los archivos que te entregamos. Es muy importante que **no le cambies el nombre a la función y que no modifiques los argumentos recibidos**. Además de los archivos entregados por nosotros, está permitido crear nuevos archivos y/o crear otras funciones si lo estimas conveniente.

Esta sección será evaluada mediante el uso de *tests*. Para poder asegurarse de la buena realización de las funciones, se podrá utilizar una serie de *tests* públicos. Es sumamente importante que **no realices la tarea basándote en los tests públicos**. Te debes basar en el enunciado y sólo puedes usar los *tests* públicos como apoyo. Debido a esto, es importante destacar que **si el alumno tiene los test públicos correctos, pero los tests privados incorrectos, no se otorgará puntaje**. En la [Sección 6: Tests](#) se explican los distintos tipos de *tests* que presenta la tarea y cómo ejecutarlos.

2. En la [Sección 5: Interfaz Gráfica e Interacción](#), se explica como implementar la parte visual del programa, el cual será desarrollada utilizando los contenidos de interfaces gráficas aprendidos en el curso. Deberás implementar las 3 ventanas ya mencionadas.

Esta parte de la tarea será corregida manualmente, por lo que es sumamente importante que **el código esté ordenado para facilitar el entendimiento**.

3. Archivos

Aunque -como se menciono anteriormente- puedes crear los archivos que quieras, en junto a este enunciado se encuentran los siguientes archivos y directorios base:

- **Modificar** `main.py`: Archivo, inicialmente vacío donde se instancian y conectan los distintos componentes del *frontend* y *backend*, permitiendo así ejecutar el programa.
- **Modificar** `backend/`: Directorio que contiene todo lo asociado al *backend* del programa. Adicionalmente, contiene los siguientes archivos:
 - **Modificar** `consultas.py`: Archivo que contiene las consultas del programa. Las funciones a implementar se explican en la [Sección 4: Programación Funcional](#).
 - **Modificar** `nodo_ligado.py`: Archivo donde debes implementar la clase `NodoLigado`. Esta clase y su uso se explica en la [Subsección 4.3: Datos de ejecución](#).
- **Modificar** `frontend/`: Directorio que contiene todo lo asociado al *frontend* del programa, es decir, todo lo relacionado a la interfaz gráfica.
- **No modificar** `data/`: Directorio que contiene una serie de archivos CSV necesarios para ejecutar la tarea. Dentro de esta carpeta habrá cuatro sub-carpetas (S, M, L y XL) que contendrán CSV de distintos tamaños.
- **No modificar** `tests_publicos/`: Directorio que contiene los tests públicos de la tarea. Estos se encargarán de revisar lo implementado en `consultas.py`.

Importante: los *tests* entregados en esta carpeta no serán los mismos que se utilizarán para la corrección de la evaluación.

- **No modificar** `utils.pyc`: Archivo que contiene la clase base `NodoLigadoAbstracto`, junto a funciones útiles para el desarrollo de la tarea. Este archivo ya se encuentra implementado.

Los archivos y directorios listados anteriormente se deberán organizar de la siguiente manera dentro de la carpeta T2/ una vez que se haya descargado toda la información necesaria:

```
T2/
├── backend/
│   ├── consultas.py
│   ├── nodo_ligado.py
│   └── ...
├── frontend/
│   ├── sprites/
│   │   ├── plasma.png
│   │   └── ...
│   └── ...
├── data/
│   ├── S/
│   ├── M/
│   ├── L/
│   └── XL/
├── tests_publicos/
│   ├── solution/
│   │   ├── test_1.py
│   │   ├── test_2.py
│   │   └── ...
│   ├── test_00_carga_habitat_objeto_correctitud.py
│   ├── test_00_carga_juguete_correctitud.py
│   └── ...
├── utils.pyc
├── main.py
├── README.md
└── .gitignore
```

4. Programación Funcional

Para poder analizar a la gran cantidad de datos que posee el *DCCornucopia*, se necesitará de tu ayuda experta para procesar la información mediante la realización de diversas consultas, por lo que deberás aplicar tus conocimientos de programación funcional.

Para esto, deberás interactuar con distintos tipos de datos, los que serán explicados con mayor detalle en la [Subsección 4.1: Datos provenientes de archivos](#) y la [Subsección 4.3: Datos de ejecución](#), y completar distintas funciones pedidas en la [Subsección 4.2: Carga de datos de `namedtuples`](#) y la [Subsección 4.4: Consultas](#), que se realizará mediante la modificación de un código pre-existente.

Cuando se menciona que algo debe estar ordenado, se entenderá de que es de menor a mayor valor, salvo que se indique lo contrario.

4.1. Datos provenientes de archivos

Para poder interactuar con los datos entregados por los clientes, tendrás que leer los archivos CSV y procesar su contenido. Para poder manejar el contenido de estos archivos se te entregan las `namedtuples` del archivo `utils.pyc`.

A modo general, los datos pueden presentar atributos que seguirán un formato específico:

- **Id:** Los datos contarán con identificadores únicos (id) que permitirán distinguir y asociar los datos. El formato de estos ids consistirá en un *integer* positivo.
- **Tiempo:** El tiempo cuando esté como *string* estará con el siguiente formato "**hh:mm**" (horas:minutos).
- **Formato de datos:** Cada archivo vendrá con los atributos correspondientes. Los atributos simples *integer* o *string*, se escriben de forma directa y se separan por comas, por ejemplo: "**Bulbasaurio,1**".

En atributos compuestos como por ejemplo *tuple*, el signo `;` separa valores dentro de un mismo atributo y el signo `:` separa los elementos de un mismo valor. Por ejemplo `((2,6),(1,5))` se presenta como "**2:6;1:5**".

Ojo que el tiempo no viene en un atributo compuesto, por lo que no habrán conflictos en la lectura.

A continuación se detallan los distintos tipos de datos y sus respectivas `namedtuples`:

4.1.1. Juguete

Indica la información de cada Juguete. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
<code>id_juguete</code>	<code>int</code>	Permite distinguir e identificar a cada juguete.	99
<code>nombre</code>	<code>str</code>	Nombre del juguete.	"Bulbasaurio"
<code>afinidades</code>	<code>Tuple[int]</code>	Indica los <code>id_afinidad</code> de los atributos con los cuales el Juguete tiene afinidad.	(1, 3)
<code>sprite</code>	<code>str</code>	Nombre del archivo del sprite sin extensión.	"001"

Ejemplo de Juguete en archivo: "**99,Bulbasaurio,1;3,001**"

Estos datos están ordenados según su `id_juguete`. La tupla `afinidades` viene ordenado por `id_afinidad`.

4.1.2. JugueteHabitat

Determina las condiciones para que aparezca un juguete. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
id_juguete	int	Permite distinguir e identificar a cada juguete.	99
id_habitat	int	Permite distinguir e identificar a cada hábitat	67
tiempo_espera	str	Tiempo a esperar para que aparezca el juguete en el hábitat	"22:30"
ids_periodo_dia	Tuple[int]	Identificador de los periodos del día en el que puede aparecer un juguete en el hábitat	(1, 2, 5, 8)

Ejemplo de JugueteHabitat en archivo: "99,67,22:30,1;2;5;8"

Ordenado por id_habitat, tiempo_espera e id_juguete, en ese orden de prioridad. Un juguete puede tener varios hábitat, y un hábitat puede tener varios juguetes. ids_periodo_dia viene ordenado por cada id_periodo_dia, de menor a mayor.

4.1.3. HabitatObjeto

Determina qué objetos son necesarios para crear un hábitat, estos objetos solo pueden ser usados para crear un hábitat, pero si son del gusto de un Juguete, le dan beneficios. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
id_habitat	int	Id del habitat que formará.	617
objetos	Tuple[Tuple[int]]	Tuplas (id_objeto, cantidad_objeto) que contienen el id del objeto y la cantidad necesaria para crear el hábitat.	((2, 1), (3, 4))

Ejemplo de HabitatObjeto en archivo: "617,2:1;3:4"

Cada habitat posee solo una forma de ser armado, ordenado por id_habitat. Objetos viene ordenado por id_objeto para cada hábitat.

4.1.4. ObjetoRecurso

Determina qué recursos se necesitan para crear un objeto, los recursos son consumidos al crear un objeto. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
id_objeto	<code>int</code>	Permite distinguir e identificar a un objeto.	176
recursos	<code>Tuple[Tuple[int]]</code>	Tuplas <code>(id_recurso, cantidad_recurso)</code> que indican los recursos y cantidades de cada uno se necesitan para crear un objeto.	<code>((76, 240), (4, 21))</code>

Ejemplo de ObjetoRecurso en archivo: **"176,76:240;4:21"**

Ordenado por `id_objeto`. Recursos está ordenado por `id_recurso`. Cada objeto posee solo una forma de ser creado.

4.1.5. RecursoRecurso

Permite crear recursos a partir de otros recursos y afinidades de **Juguete**, consumiendo los recursos. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
id_recurso	<code>int</code>	Id del recurso resultante.	529
recursos	<code>Tuple[Tuple[int]]</code>	Tuplas <code>(id_recurso, cantidad_recurso)</code> que indican los recursos y cantidades de cada uno necesarias para crear el recurso .	<code>((12, 2), (44, 5))</code>
id_afinidad	<code>int</code>	Afinidad necesaria para crear el recurso.	8

Ejemplo de RecursoRecurso en archivo: **"529,12:2;44:5,8"**

Ordenado por `id_recurso` de mayor a menor (para ambos **RecursoRecurso** y las tuplas). Cada **recursos** posee una sola forma de ser creado mediante esta tabla (salvo el recurso de menor id, que no posee forma de ser creado), pero puede ser creado también por **Juguete**. El id del recurso resultante siempre será mayor a todos los `id_recurso` que permiten su creación.

Cuando se mencione `generador_recurso_recurso` o `gen_recurso_recurso`, sus datos vendrán siguiendo el mismo orden que el especificado en esta sección.

4.1.6. JugueteRecurso

El recurso que puede generar un **Juguete** naturalmente. Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
id_juguete	<code>int</code>	Permite distinguir e identificar a cada juguete.	33
id_recurso	<code>int</code>	Permite distinguir e identificar a cada recurso.	767
tiempo_espera	<code>str</code>	Determina cuánto tiempo debe pasar para crear el recurso	"12:06"
cantidad	<code>int</code>	Determina cuánto recurso se genera.	112

Ejemplo de JugueRecurso en archivo: "33,767,12:06,112"

Ordenado por `id_juguete`. Hay solo un recurso por juguete, pero puede haber más de un juguete por recurso

4.1.7. JugueteObjeto

Determina qué objetos son los favoritos de un juguete. Si un objeto favorito esta presente, da bonificación a cada juguete que lo tenga como favorito, lo que permite que genere más recursos en menos tiempo. Solo se consideran los objetos distintos, por lo que si un objeto aparece mas de una vez, da la misma bonificación que solo uno. La cantidad final producida será de:

$$cantidad \times 1,1^{favoritos}$$

Donde:

- *cantidad* es la cantidad de recursos que el Juguete produce por defecto.
- *favoritos* es la cantidad de objetos favoritos distintos del juguete que están presentes.

Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
<code>id_juguete</code>	<code>int</code>	Permite distinguir e identificar a cada juguete.	582
<code>id_objeto</code>	<code>int</code>	Permite distinguir e identificar a cada objeto.	7

Ejemplo de JugueteObjeto en archivo: "582,7"

Ordenado por (`id_juguete`, `id_objeto`). Pueden haber varios objetos para un juguete y varios juguetes para un objeto, pero el par no se repite.

4.1.8. PeriodoDia

Representan los periodos de un día, su orden es el mismo que el presente en los datos.

Posee los atributos:

Atributo	Tipo	Descripción	Ejemplo
<code>id_periodo_dia</code>	<code>int</code>	Identificador del periodo del día.	567
<code>nombre</code>	<code>str</code>	Nombre de cada periodo del día.	"Mañana"
<code>duracion</code>	<code>str</code>	Indica la duración de este periodo	"02:10"

Se entiende el fin de periodo día como el instante del inicio del siguiente. Para ese momento, se considera como que el periodo día actual es el que inicia en ese minuto.

Ejemplo de PeriodoDia en archivo: "567,Mañana,02:10"

Ordenado por `id_periodo_dia`

4.2. Carga de datos de `namedtuples`

En la carpeta “`data/`” podrás encontrar cuatro sub-carpetas (S, M, L y XL) con bases de datos de distintos tamaños: pequeños, medianos, grandes y extra-grandes, respectivamente. Deberás copiar esta carpeta y ubicarla en tu carpeta de la tarea T2/, siguiendo la estructura indicada en [Sección 3: Archivos](#). Para poder trabajar las consultas de esta tarea deberás cargar la información en **generadores** que contengan las `namedtuple` definidas en `utils.pyc`. La información de estos **generadores** se obtendrá a partir de archivos de extensión `.csv` que siguen el mismo formato de las `namedtuple` mencionadas en la sección anterior. Para asegurar el correcto funcionamiento de estas funciones durante la corrección de la tarea y ejecución de los tests, las funciones no deben modificar los paths recibidos de estas funciones.

La carga de información requiere que implementes las siguientes funciones que definen **generadores**. Cada función recibirán un `str path` correspondiente a la ruta del archivo `.csv` que le corresponde y deberá retornar un **generador** con instancias de la `namedtuple` usando la información del archivo.

```
Modificar def cargar_juguete(path: str) -> Generator[Juguete]
```

```
Modificar def cargar_juguete_habitat(path: str) -> Generator[JugueteHabitat]
```

```
Modificar def cargar_habitat_objeto(path: str) -> Generator[HabitatObjeto]
```

```
Modificar def cargar_objeto_recurso(path: str) -> Generator[ObjetoRecurso]
```

```
Modificar def cargar_recurso_recurso(path: str) -> Generator[RecursoRecurso]
```

```
Modificar def cargar_juguete_recurso(path: str) -> Generator[JugueteRecurso]
```

```
Modificar def cargar_juguete_objeto(path: str) -> Generator[JugueteObjeto]
```

```
Modificar def cargar_periodo_dia(path: str) -> Generator[PeriodoDia]
```

Esta tarea contendrá *tests* que evaluarán el óptimo de la solución implementada, por lo que estas funciones serán usadas para cargar y crear los generadores que recibirán las consultas definidas en la siguiente sección. Por lo tanto, se recomienda **completar primero estas funciones antes de completar y probar cualquier consulta**.

4.3. Datos de ejecución

Los **datos de ejecución** representan el estado del juego, ya que en ellos se registran los inventarios, los tiempos de producción de los juguetes, etc. Esto quiere decir que no provienen de los datos base.

Para manejar esta información de forma eficiente y ordenada deberás utilizar una estructura de **NodoLigado**.

4.3.1. Clase NodoLigado

Se debe implementar la clase `NodoLigado` en el archivo entregado `nodo_ligado.py`, la cual hereda de la clase base `NodoLigadoAbstracto` del módulo `utils.pyc`. Esta clase permite organizar los distintos **datos de ejecución** (como juguetes productores, recursos o hábitats) en cadenas independientes, asegurando que se mantengan siempre ordenados de forma ascendente según su `id`.

No modificar `class NodoLigadoAbstracto:`

- No modificar `def __init__(self, id: int, siguiente: NodoLigado | None = None, **kwargs):`

El nodo ligado abstracto recibe los atributos a guardar como atributos de la instancia, estos pueden ser `id` y `siguiente`, pero se permiten adicionales mediante `kwargs`. El parametro `siguiente` corresponde al `NodoLigado` que venga después del actual, es `None` si no existe.

Los métodos que deberás modificar en la clase `NodoLigado` son:

No modificar `class NodoLigado(NodoLigadoAbstracto):`

- Modificar `def insertar(self, other) -> NodoLigado:`
Inserta un nuevo objeto de tipo `NodoLigado` en la cadena correspondiente, manteniendo el orden ascendente por `id`. Retorna el primer nodo de la cadena resultante.
- Modificar `def remover(self, id) -> NodoLigado | None:`
Saca de la cadena el nodo que coincida con el ID entregado y la reconecta. Retorna el primer nodo de la cadena resultante, o `None` si esta queda vacía.
- Modificar `def buscar(self, id) -> NodoLigado | None:`
Recorre la cadena y retorna el objeto con el ID solicitado. Si el ID no se encuentra, debe retornar `None`.

Solo debes modificar estas tres funciones para nodo ligado y puedes asumir que las tres funciones serán siempre llamadas desde el primer nodo de la cadena de `Nodo Ligado`. Recuerda que deberás preocuparte de que el orden por `id` se mantenga. Puedes asumir que el nodo que se inserta usando `insertar` no está conectado a otro nodo, es decir: `nodo_a_insertar.siguiente = None`.

Cabe recalcar que método (`__init__`) ya viene implementado en la clase base y **no debe ser modificado**.

4.3.2. Hábitat

Representa un hábitat construido que está a la espera de atraer a un juguete. Es una instancia de `NodoLigado`. Un hábitat sólo puede atraer a juguetes que aún no estén presentes.

Atributo	Tipo	Descripción	Ejemplo
id	int	Corresponde al <code>id_habitat</code> del hábitat construido.	67
tiempo_presente	int	Contador de minutos que determina la aparición de un juguete; cuando este valor alcanza el tiempo de espera requerido, el juguete aparece.	45

En caso de que dos juguetes aparezcan en el mismo tiempo, el juguete de menor ID aparece primero.

4.3.3. JugueteProductor

Representa a un juguete que ya se encuentra en el refugio y está generando recursos. Es una instancia de `NodoLigado`.

Atributo	Tipo	Descripción	Ejemplo
id	int	Corresponde al <code>id_juguete</code> del juguete.	7
tiempo_actual	int	Contador de minutos que lleva el juguete preparando la producción de un recurso. Regresa a 0 al generar su recurso	10

4.3.4. Objeto

Representa la cantidad de objetos fabricados. Al igual que las entidades anteriores, es una instancia de `NodoLigado`.

Atributo	Tipo	Descripción	Ejemplo
id	int	Corresponde al <code>id_objeto</code> del objeto.	176
cantidad	int	Unidades que han sido creadas.	5

4.3.5. Recurso

Representa el stock disponible de un recurso. Es una instancia de `NodoLigado`.

Atributo	Tipo	Descripción	Ejemplo
id	int	Corresponde al <code>id_recurso</code> del recurso.	529
cantidad	int	Unidades totales del recurso en el inventario.	100

4.4. Consultas

Utilizando los datos almacenados en generadores deberás procesar su información mediante las siguientes consultas.

4.4.1. Consultas que solo usan datos de archivos

```
Modificar def ahora_es(generator_periodo_dia: Generator, minutos: int) -> tuple:
```

Función que recibe un **generador** con instancias de `PeriodoDia`, y un `int` con la cantidad de minutos totales que han pasado desde el inicio.

Retorna una **tupla** con la información del día, hora y periodo específico que se encuentra actualmente, donde cada tupla tiene la forma:

```
(dia_actual: int, hora_actual: str, nombre_periodo_dia: str,
 hora_inicio_periodo_dia: str, hora_fin_periodo_dia: str)
```

Por ejemplo: (0, "01:30", "madrugada", "00:00", "06:00")

```
Modificar def recursos_a_partir_de_recurso(generator_recurso_recurso: Generator,
 id_recurso: int) -> Generator:
```

Función generadora que recibe un **generador** con instancias de `RecursoRecurso`, y un `int` con el identificador del recurso que servirá como base para la fabricación.

Retorna un **generador** ordenado por el `id` del recurso resultante, donde cada elemento es una **tupla** con la información de los materiales fabricables de la forma:

```
(id_recurso_creadable: int, afinidades_necesarias: tuple, cantidad_requerida: int)
```

Donde `afinidades_necesarias` es una **tupla** con todas las afinidades requeridas para la creación, ordenada por `id_afinidad`.

```
Modificar def juguetes_producen(generator_juguete_recurso: Generator,
 id_recurso: int) -> Generator:
```

Función generadora que recibe un **generador** con instancias de `JugueteRecurso`, y un `int` que representa el identificador del recurso solicitado.

Retorna un **generador** que contiene los `id_juguete` de los juguetes capaces de producir dicho recurso. Los elementos deben estar ordenados de menor a mayor sin que se repita ningún identificador.

```
Modificar def juguetes_productivos(generator_juguete_recurso: Generator,
 minimo: float | None = None) -> Generator:
```

Función generadora que recibe un **generador** con instancias de `JugueteRecurso`, y un `float` (o `None`) que actúa como umbral de productividad

Retorna un **generador** con los `id_juguete` cuya productividad sea estrictamente mayor al valor `minimo` entregado.

Donde la **productividad** se calcula como la cantidad de recursos que un juguete genera dividida por su tiempo de espera.

Si el parámetro `minimo` es `None`, la función debe buscar y retornar **únicamente** a los juguetes que tengan la productividad más alta de toda la base de datos (en caso de empate, se entregan todos los que compartan ese valor máximo).

Modificar

```
def habitats_de_interes(generator_juguete: Generator,  
    generator_juguete_habitat: Generator) -> Generator:
```

Función generadora que recibe un **generador** con instancias de **Juguete** (que representa los juguetes que ya han aparecido) y un **generador** con instancias de **JugueteHabitat**.

Retorna un **generador** con los `id_habitat` de los hábitats de interés.

Se considera que un hábitat es “de interés” cuando en él aun pueden aparecer juguetes nuevos.

Modificar

```
def cadena_creacion(generator_juguete_recurso: Generator,  
    generator_recurso_recurso: Generator,  
    id_recurso: int) -> dict:
```

Recibe un **generador** de **JugueteRecurso**, un **generador** de **RecursoRecurso** y el id del recurso que se desea saber cómo se arma.

Esta consulta permite visualizar la totalidad de cómo se puede armar un **recurso**, desglosando el cómo se obtiene, ya sea los juguetes que lo producen como qué recursos lo forman.

Retorna un **diccionario** que representa la cadena de producción completa. Debe incluir el **Recurso** objetivo y todos los requisitos directos e indirectos para su creación.

Donde cada nivel del **diccionario** sigue la estructura:

```
{id_recurso: {"recursos": Dict, "juguetes": Tuple}}
```

Si un recurso es no necesita otros materiales para crearse, su **diccionario** de recursos estará vacío.

Por ejemplo, si quisiéramos ver la cadena del recurso con id 10 , el **diccionario** se vería de la siguiente manera:

```
1 {  
2   10: {  
3     "recursos": {  
4       2: {  
5         "recursos": dict(),  
6         "juguetes": (2, 3, 5)  
7       },  
8       9: {  
9         "recursos": {  
10          1: { "recursos": dict(), "juguetes": (1, ) }  
11        },  
12        "juguetes": tuple()  
13      }  
14    },  
15    "juguetes": (1, 3)  
16  }  
17 }
```

Modificar

```
def juguetes_a_aparecer(generator_juguete_habitat: Generator,  
    generator_periodo_dia: Generator, id_habitat: int,  
    momento_dia: int = 0) -> Generator:
```

Función generadora que recibe un **generador** de **JugueteHabitat**, un **generador** de **PeriodoDia**, un **int** con el `id_habitat` y un **int** con el `momento_dia` actual.

Retorna un **generador** con la información de los juguetes que podrían aparecer en el habitat con id `id_habitat`, donde cada elemento es una **tupla** de la forma:

`(id_juguete: int, tiempo_de_espera: int)`

Donde el tiempo de espera son los minutos que tardarían en aparecer desde el minuto actual. Las tuplas deben estar ordenadas por orden de aparición y, en caso de empate, por `id_juguete`.

```
Modificar def juguetes_autosuficientes(generator_juguete: Generator,
                                     generator_juguete_objeto: Generator,
                                     generator_objeto_recurso: Generator,
                                     generator_juguete_recurso: Generator,
                                     generator_recurso_recurso: Generator) -> Generator:
```

Función generadora que recibe cinco **generadores** correspondientes a: `Juguete`, `JugueteObjeto`, `ObjetoRecurso`, `JugueteRecurso` y `RecursoRecurso`.

Retorna un **generador** que entrega, ordenadamente y sin repetir, los `id_juguete` de los juguetes **autosuficientes**.

Un juguete se considera **autosuficiente** si tiene la capacidad de obtener, por sus propios medios, la totalidad de los **recursos** necesarios para fabricar al menos uno de sus **objetos favoritos**.

Un recurso se considera “**obtenible**” por el juguete si:

- El juguete lo produce de forma natural
- El juguete puede fabricarlo por su propia cuenta a partir del recurso que produce naturalmente

```
Modificar def habitat_eventualmente_creables(generator_juguete: Generator,
                                             generator_juguete_recurso: Generator,
                                             generator_recurso_recurso: Generator,
                                             generator_objeto_recurso: Generator,
                                             generator_habitat_objeto: Generator) -> Generator:
```

Función generadora que recibe cinco **generadores** correspondientes a: `Juguete` (juguetes presentes), `JugueteRecurso`, `RecursoRecurso`, `ObjetoRecurso` y `HabitatObjeto`.

Entrega un **generador** con los `id_habitat` de los hábitats que puedan ser construidos con lo contenido en los generadores, considerando que los juguetes ya han aparecido.

4.4.2. Consultas que usan datos de ejecución

```
Modificar def avanzar_produccion(generator_juguete_recurso: Generator,  
                                generator_juguete_objeto: Generator, JugueteProductor: NodoLigado | None,  
                                objetos: set, minutos: int = 1) -> Generator:
```

Función generadora cuyos parámetros son dos **generadores** correspondientes a: **JugueteRecurso** y **JugueteObjeto**, **JugueteProductor** son los juguetes que producen los recursos, **objetos** son los objetos ya existentes y **minutos** es cuánto tiempo avanzar en la producción

Entrega un **generador** con **tuplas** (**id_recurso**, **cantidad_recurso**), ordenados por **id_recurso**, apareciendo sólo una vez por recurso creado con la cantidad creada del mismo.

Donde **minutos** representa el tiempo que se debe avanzar para determinar los recursos creados.

```
Modificar def crear_recursos(generator_juguete_recurso: Generator,  
                             generator_juguete_objeto: Generator) -> Generator:
```

Función generadora cuyos parámetros son dos **generadores**: **JugueteRecurso** y **JugueteObjeto**.

Retorna distintos valores según el que recibe, según la siguiente tabla de comportamiento respecto al valor recibido:

- **None**, actúa como si se recibiese un 1 para avanzar el tiempo.
- **int**, avanza la cantidad que indica el **int** y entrega una tupla con pares ordenados por **id_recurso** de (**id_recurso**, **cantidad_recurso**) para recursos y cantidades creados en esa cantidad de tiempo. El valor será mayor o igual a 1.
- **Juguete**, incluye al **Juguete** a la Cadena de **NodoLigado** de **JugueteProductor**, sin avanzar el tiempo. No entrega un valor.
- **set**, incluye lo contenido en el **set** en el set interno **set_objetos**. No entrega un valor.
- **"end"**, el generador entrega una tupla con la Cadena de **NodoLigado** de **JugueteProductor** y el set interno **set_objetos**. Posteriormente finaliza.

```
Modificar def agregar_recursos(recurso: NodoLigado | None,  
                               nuevos_recursos: Tuple[Tuple[int, int]]) -> NodoLigado:
```

Función cuyos parámetros son un **recurso** que es el primero de una cadena de **Recurso**, y una **tupla** de la forma (**id_recurso**, **cantidad_recurso**) que indica qué recursos agregar a la cadena.

Retorna primer **Recurso** de la nueva cadena, agregando la cantidad al **Recurso** en caso de que ya haya estado presente en la cadena, y en el otro caso agregándolo a la cadena.

```
Modificar def por_aparecer(generator_juguete: Generator,  
                           generator_juguete_habitat: Generator, generator_periodo_dia: Generator,  
                           Habitat: NodoLigado | None, momento_dia: int=0) -> Generator:
```

Función generadora que recibe un generador con instancias **Juguete** que ya han aparecido, un generador de **JugueteHabitat**, un generador de **PeriodoDia**, un **NodoLigado** de los **Habitat** ya existentes, y el **momento_dia** actual.

Retorna un **generador** con los **ids** de los **Juguetes** que aparecerán en cada uno de los **habitats**, usando el orden de los **Habitat**. Si en un **Habitat** un juguete no aparece, se entrega el valor **None** en esa posición. Si un mismo juguete debiese aparecer en dos **hábitat** al mismo tiempo, se considera que aparece en el de menor **id**.

Si un juguete ya debió haber aparecido en el hábitat (`Habitat.tiempo_presente` es mayor al tiempo de aparición del `Juguete`), no se considera como que aparecerá.

```
Modificar def habitat_creables(generator_juguete: Generator,
                                generator_juguete_recurso: Generator,
                                generator_recurso_recurso: Generator,
                                generator_objeto_recurso: Generator,
                                generator_habitat_objeto: Generator,
                                Recurso: NodoLigado | None) -> Generator:
```

Función generadora cuyos parámetros son 5 **generadores** (el primero conteniendo los juguetes presentes) y un `NodoLigado` que posee los recursos existentes.

Retorna un **generador** con los hábitats que pueden crearse sin avanzar el tiempo.

```
Modificar def manejo_habitat(gen_juguete: Generator, gen_juguete_habitat: Generator,
                              gen_habitat_objeto: Generator, gen_objeto_recurso: Generator,
                              gen_recurso_recurso: Generator, gen_periodo_dia: Generator,
                              tiempo_inicial: int = 0) -> Generator:
```

Función generadora cuyos parámetros son 6 **generadores** y un `int`.

Internamente almacena los `Habitat` que aún no han sido habitados, los `Recurso` que son los recursos disponibles, y los `Objeto` existentes.

Retorna distintos valores al simular el manejo de hábitats en base a los valores que reciba, según la siguiente listas de comportamientos:

- `None`, entrega una tupla de los ids de los `Juguetes` que aparecen. Luego avanza una unidad de tiempo a su tiempo actual.
- `tupla_data_recursos`: `Tuple` (tupla que contiene pares de (`id_recurso`, `cantidad`)), agrega la información de los recursos al `NodoLigado` interno de `Recurso`. No entrega un valor.
- `"crear"`, intenta de crear cada hábitat de interés desde el menor `id_habitat` (una vez por hábitat de interés). Luego entrega una tupla con dos elementos, primero el set de los `id_objeto` de los `Objetos` que fueron creados para crear los `Habitat`, y luego una tupla de los `id_habitat` que fueron creados.

Para los **hábitat de interés** deberás considerar los `Juguetes` que van a aparecer como que ya aparecieron, para evitar crear `Habitat` innecesariamente.

- `"report"`, retorna una tupla de cinco elementos: primero tiempo actual, luego el set de los ids de `Juguete` creados y finalmente tres `NodoLigado`: (`Habitat`, `Objeto`, `Recurso`) donde:
 - `Habitat` es la `Cadena de NodoLigado` de los hábitats que han sido creados. Es `None` si no hay hábitats que han sido creados.
 - `Objeto` y `Recurso` son `Cadena de NodoLigado` de los objetos y recursos, respectivamente. Serán `None` si su cadena está vacía.

4.4.3. Consulta de simulación

```
Modificar def simular(gen_juguete: Generator, gen_juguete_habitat: Generator,  
    gen_habitat_objeto: Generator, gen_objeto_recurso: Generator,  
    gen_recurso_recurso: Generator, gen_juguete_recurso: Generator,  
    gen_juguete_objeto: Generator, gen_periodo_dia: Generator) -> Generator:
```

Recibe 8 **generadores**, y usa de tiempo inicial 0 para los minutos. Luego simula DCCornucopia funcionando de distintas formas en base al valor que reciba, según la siguiente tabla de comportamientos:

- **None**, avanza una unidad de tiempo
- **"next"**, avanza hasta la unidad de tiempo del siguiente **evento de interés**.
- **"end"**, avanza hasta que no queden más **eventos de interés**.

Por cada unidad de tiempo que avanza, realiza estas acciones en el siguiente orden:

1. Revisa si en los hábitats vacíos aparece un Juguete
2. Generar los recursos provenientes de Juguetes
3. Crea hábitats de interés usando los recursos y objetos actuales.

Se da la simulación por terminada si no se pueden generar más hábitats de interés, y todos los hábitats presentes poseen un juguete.

Se clasifican como **eventos de interés**:

- Se crea un hábitat de interés.
- Aparece un juguete.

El generador retorna un **generador** de **eventos de interés** el cual contiene (**tiempo_actual**: **int**, ***evento**) , donde **evento** está compuesto por pares (**tipo_evento**, **dato_evento**) tales que pueden ser:

- (**"juguete"**, Juguete), para cuando un juguete llega a un hábitat.
- (**"habitat"**, id_habitat), para cuando se crea un hábitat.

En caso de empate de tiempo, vendrán primero los de **"juguete"** por orden de **id_juguete**, y luego los de **"habitat"** por orden de **id_habitat**.

5. Interfaz Gráfica e Interacción

Con el fin de administrar correctamente la información de los juguetes y visualizar el comportamiento del sistema, se ha pedido diseñar una interfaz gráfica que facilite la interacción y muestre los resultados de la simulación de manera clara.

5.1. Modelación del programa

En esta sección se evaluarán los siguientes aspectos generales:

- Correcta **modularización** del programa, esto quiere decir que se debe respetar y seguir una adecuada estructuración entre *frontend* y *backend*, con un **diseño cohesivo** y de **bajo acoplamiento**.
- Correcto uso de señales entre *backend* y *frontend*.
- Un flujo prolijo a lo largo del programa. Esto quiere decir que el usuario puede navegar sin problemas entre las distintas partes que componen *DCCornucopia* solo ejecutando una vez el programa. En otras palabras, un usuario nunca debe quedarse atorado en alguna parte del programa que lo obliga a cerrar *DCCornucopia* y volver a ejecutarlo. A modo de ejemplo: si un usuario ingresa a una sección y no se puede mover a otra sección, esto es considerado como una mala implementación.
- Una interfaz interactiva integrada con las funcionalidades pedidas. Esto quiere decir que se espera que la comunicación y el comportamiento del programa sea visible y controlable desde la interfaz. **No se asignará puntaje** si las funcionalidades solicitadas no son visibles en la interfaz ó si es que estas solo se pueden comprobar en la consola o en el código, a menos que se indique explícitamente lo contrario.
- Generación de las ventanas mediante código programado por el estudiante. Es decir, **no se permite** la creación de ventanas con el apoyo de herramientas como QtDesigner, QML, entre otros.

5.2. Ventanas

Para la correcta implementación de *DCCornucopia*, se espera la creación de **tres ventanas**, cada una con elementos mínimos de interfaz que deben estar presentes, los cuales serán detallados a continuación.

Importante:

Los esquemas y diseños que serán expuestos son únicamente referenciales. No es necesario que tu tarea sea una copia exacta de estos; por lo que **lo único que será evaluado es que los elementos mínimos estén presentes y funcionen del modo que se exige**.

Los elementos mínimos de cada ventana serán explicitados en la sección correspondiente; pero en ningún caso se les exige cosas relacionadas a la estética de la interfaz gráfica. Por lo tanto, una ventana que sólo tenga los elementos expuestos en los esquemas, tiene el mismo nivel de validez que otra llena de decoraciones y efectos interactivos, esto, suponiendo que ambas tengan el comportamiento deseado. Parte del comportamiento deseado incluye el poder controlar el tamaño de estas, tal que se puedan usar en pantallas de cualquier tamaño.

En caso de que lo consideres necesario puedes ubicar los elementos en otras posiciones; agregar creatividad inventando botones nuevos; nuevas interacciones, diseños y hasta animaciones. Cabe destacar que si se desea desarrollar funcionalidades extras, estas serán evaluadas bajo los mismos criterios generales mencionados anteriormente. Esto quiere decir que, si el programa falla debido a una funcionalidad extra, se aplicara el descuento en el *item* correspondiente.

5.2.1. Ventana de Inicio

Esta es la primera ventana en visualizarse al ejecutar el programa. Se muestra un mensaje de bienvenida, dos selecciones de carpetas y **dos botones**. La selección de las carpetas es mediante un `QFileDialog` por cada uno, el primero de los datos y el segundo de los sprites. Un botón para acceder al **Índice de Juguetes**, y otro para acceder a la **Simulación de Juego**.

En la [Figura 1](#) se muestra un ejemplo de cómo puede verse esta ventana.

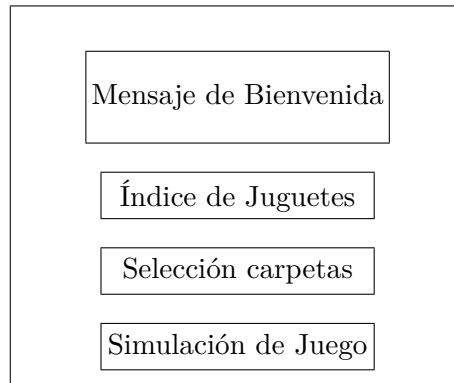


Figura 1: Ventana de Inicio

5.2.2. Ventana de Índice de Juguetes

El objetivo de esta ventana es visualizar el catálogo completo de juguetes disponibles. Para esto, se debe cargar la información desde un archivo de datos y una carpeta de sprites. Los elementos mínimos de esta ventana son:

- Se debe utilizar un `QGridLayout` *scrollable* para organizar y visualizar los juguetes en forma de grillas. Cada celda de la grilla debe mostrar, de forma clara, el **ID**, el **nombre** y el **sprite** correspondiente al juguete.
- **Botón “Regresar”** que se encarga de volver a la Ventana de Inicio.

En la [Figura 2](#) se muestra un ejemplo de cómo puede verse la ventana índice de juguetes, con los elementos previamente mencionados.

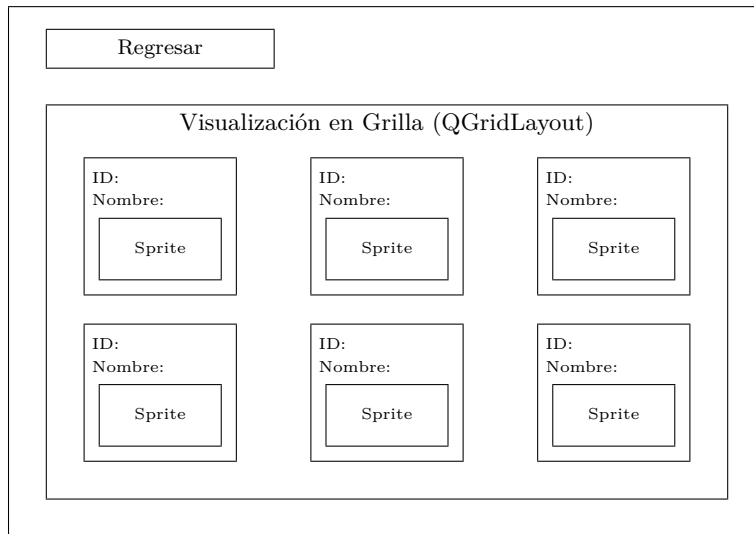


Figura 2: Ventana de Índice de Juguetes

5.2.3. Ventana de Simulación de juego

Esta ventana permite ejecutar y observar el comportamiento del juego. El estado inicial se construye a partir de una carpeta donde cada archivo representa los Tipos de Datos Iniciales.

Los elementos mínimos que deben estar presentes en esta ventana son los siguientes:

- **Visualización de Juguetes:** Se debe utilizar un **QGridLayout** *scrollable* para mostrar los juguetes que se encuentran presentes en el refugio. Cada celda de la grilla debe mostrar, de forma clara, el **ID**, el **nombre** y el **sprite** correspondiente al juguete.
- **Información de Tiempo:** La interfaz debe mostrar datos de tiempo, incluidos el día correcto, la hora y el periodo actual (ej. “Día 1 (22:33) Noche (21:00-23:59)”).
- **Información de Hábitats vacíos:** La interfaz debe mostrar la cantidad de hábitats vacíos que se encuentran actualmente.
- **Controles de Simulación:** Deben incluirse tres botones interactivos para gestionar el flujo temporal del juego:
 - **Step forward (▶):** Avanza una unidad de tiempo.
 - **Fast forward (⏩):** Avanza hasta el siguiente evento de interés.
 - **Checkered flag (🏁):** Avanza la simulación hasta que no ocurran más eventos de interés.

Los comportamientos de los **controles de simulación** para los tres casos se encuentran explicados en [Subsubsección 4.4.3: Consulta de simulación](#).

- **Registro de Eventos:** Se debe implementar un **QTextBrowser** desplazable (*scrollable*) que muestre los eventos de interés que ocurren durante la simulación.
- **Botón de Regreso:** Un botón “Regresar” que permita volver a la Ventana de Inicio.

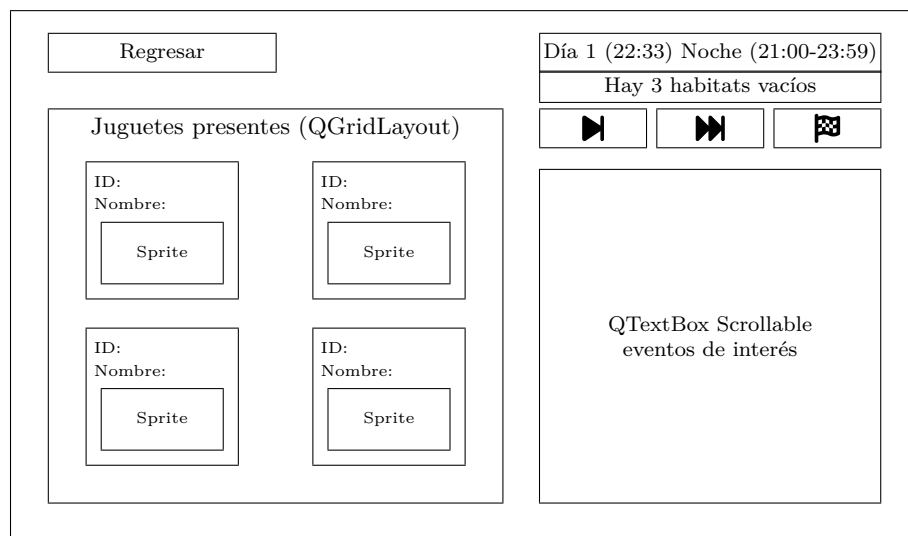


Figura 3: Ventana de Simulación de juego

6. Tests

El objetivo principal de la implementación de los *tests* es la eficiencia. Cada test posee un tiempo límite de ejecución permitido. Estos tiempos pueden ser distintos para distintos *tests*. **Si no se cumple con el tiempo límite, no se otorgará puntaje.**

Los *tests* presentes en esta evaluación se dividirán en dos conjuntos:

- **Correctitud:** Evaluará el comportamiento de la solución implementada con respecto a posibles casos bordes. En estos casos, se probará la función con generadores ya cargados por el programa.
- **Carga de datos:** Evaluará el comportamiento de la solución implementada con respecto a la eficiencia del código y su capacidad de trabajar con archivos de diversos tamaños.

Para lograr este objetivo de esta evaluación, es esencial un correcto uso de **generadores** y **programación funcional**. El no aplicar correctamente los contenidos anteriormente mencionados, puede provocar que las funciones no terminen en el tiempo esperado.

Reiterando lo indicado en el [Flujo del programa](#): Es sumamente importante que **no realices la tarea basándote en los tests públicos**. Te debes basar en el enunciado y sólo puedes usar los tests públicos para apoyo. Debido a esto, es importante destacar que **si el alumno tiene los test públicos correctos, pero los tests privados incorrectos, no se otorgará puntaje.**

Adicionalmente, para los *tests* de Carga de datos de la sección [Carga de datos de namedtuples](#) se evalúa el uso de memoria máxima de la función para comprobar el uso de programación funcional. Para tener puntaje en esta se requiere que el test haya pasado su equivalente de los *tests* de Correctitud de la misma sección.

6.1. Ejecución de tests

Para la corrección automática se entregarán varios archivos `.py` los cuales contienen diferentes *tests* que ayudan a validar el desarrollo de la tarea. Para ejecutar estos *tests*, primero debes posicionar tu terminal/console en la carpeta de la tarea `Tareas/T2/`. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests*:

- `python3 -B -m unittest discover tests_publicos -v -b`

En cambio, si deseas ejecutar un subconjunto de *tests*, puedes hacerlo escribiendo lo siguiente:

- `python3 -B -m unittest -v -b tests_publicos.<test_N>`
Reemplazando `<test_N>` por el nombre del archivo de test que desees probar.

Por ejemplo, si quisieras probar si realizaste correctamente la función “`ahora_es`”, deberás escribir lo siguiente:

```
python3 -B -m unittest -v -b tests_publicos.test_01_ahora_es_correctitud
```

Importante: Recuerda que para ejecutar Python debes usar el comando específico de tu computador, este puede ser: `py`, `python`, `py3`, `python3` o `python3.12`.

7. `utils.pyc`

Se entregará un archivo `.pyc`, el cual contendrá las `namedtuples` y `NodoLigadoAbstracto` que deben utilizar para el desarrollo de la tarea. Estas `namedtuples` y `NodoLigadoAbstracto` solo deben ser importadas y utilizadas, sin la posibilidad de ser modificadas o ver su funcionamiento.

7.1. `Namedtuples`

Tal como se indica en la [Subsección 4.1: Datos provenientes de archivos](#), cada uno de los datos entregados en los archivos CSV presenta una `namedtuples` que lo representa. El detalle de estas `namedtuples` se encuentra en la [Subsección 4.1](#).

7.2. `NodoLigadoAbstracto`

Adicionalmente, para que puedas implementar `NodoLigado` se te entregan la clase auxiliar detallada en la sección [Subsubsección 4.3.1](#).

8. `.gitignore`

Para esta tarea **deberás utilizar un `.gitignore`** para ignorar los archivos indicados, este deberá estar dentro de tu carpeta `Tareas/T2/`.

Los elementos que no debes subir y **debes ignorar mediante el archivo `.gitignore`** para esta tarea son:

- El enunciado.
- La carpeta `data/` y los archivos `csv` correspondientes a los datos.
- La carpeta `test_publicos/`.
- La carpeta `sprites` dentro de la carpeta `frontend`.
- Cualquier archivo `utils.pyc` o algún otro que posea dicha extensión.

Recuerda **no ignorar archivos vitales de tu tarea como los que tú creas o modificas, o tu tarea no podrá ser revisada**.

El correcto uso del archivo `.gitignore`, implica que los archivos **deben** no subirse al repositorio debido al uso archivo `.gitignore` y no debido a otros medios.

Importante Debes asegurarte de que ni los archivos de datos ni los archivos de los `tests` no sean subidos a tu repositorio personal, en caso contrario, se aplicarán 10 décimas de descuento de formato en la corrección de la evaluación. Dado que en esta evaluación presenta archivos de gran tamaño, junto a los archivos base de la tarea se incluye un archivo `.gitignore` que ignora todos los archivos `csv`.

Finalmente, en caso hacer `commit` de la carpeta `“data/”`, `“tests_publicos/”` o `“sprites/”`, debido al tamaño de dichos archivos, deben deshacer este cambio. En caso de que lleguen a enfrentarse a este problema, deben:

1. Mover los datos de las carpetas fuera del repositorio.
2. Hacer `commit` y `push` de los cambios. Esto es, haciendo `add` de los archivos eliminados. Si hacen `“add .”` desde la carpeta que desean eliminar el contenido, funcionará correctamente.
3. Devolver los datos a su repositorio

9. Importante: Corrección de la tarea

En el [siguiente enlace](#) se encuentra la distribución de puntajes. Para esta tarea, el carácter funcional del programa será el pilar de la corrección, es decir, **sólo se corrigen tareas que puedan ejecutar**. Por lo tanto, se recomienda hacer periódicamente pruebas de ejecución de su tarea y *push* en sus repositorios, corroborando que cada *test* que les pasamos para cada consulta corra en el tiempo indicado ([Sección 6: Tests](#)), en caso contrario se asumirá un resultado incorrecto.

Importante: Todo ítem corregido automáticamente será evaluado de forma ternaria: puntaje completo si pasa todos los *tests* de dicho ítem, medio punto para quienes pasan más del 70 % de los *test* de dicho ítem, y 0 puntos para quienes no superan el 70 % de los *tests* en dicho ítem. Finalmente, todos los descuentos serán asignados automáticamente por el cuerpo docente.

La corrección se realizará en función del último *commit* realizado antes de la fecha oficial de entrega (viernes 15 de mayo a las 20:00). Es necesario rellenar el formulario de utilización de IA, se haya utilizado o no, que tiene fecha límite el sábado 16 de mayo a las 20:00.

Para terminar, si durante la realización de tu tarea se te presenta algún problema o situación que pueda afectar tu rendimiento, no dudes en contactar al ayudante de Bienestar de tu sección. El correo está en el [siguiente enlace](#).

10. Restricciones y alcances

- Esta tarea es **estrictamente individual**, y está regida por el [Código de honor de Ingeniería](#).
- Tu programa debe ser desarrollado en Python 3.12.X con X mayor o igual a 9.
- Esta tarea no permite el uso de `getattr`, `setattr` ni `hasattr`.
- Tu programa debe estar compuesto por uno o más archivos de extensión `.py` que estén correctamente ordenados por carpeta. **No se revisará archivos en otra extensión como `.ipynb`.**
- Todo el código entregado debe estar contenido en la carpeta y rama (*branch*) indicadas al inicio del enunciado. Ante cualquier problema relacionado a esto, es decir, una carpeta distinta a **Tareas/T2/** o una rama distinta a **main**, se recomienda preguntar en las [discussions del foro](#).
- Si no se encuentra especificado en el enunciado, supón que el uso de cualquier librería Python está prohibida. Pregunta en la *discussion* especial del [foro](#) si es que es posible utilizar alguna librería en particular.
- Debes adjuntar un **único archivo markdown**, llamado `README.md`, **conciso y claro**, donde describas las referencias a código externo. El no incluir este archivo, incluir un readme vacío o el subir más de un archivo `.md`, conllevará un [descuento](#) en tu nota.
- Esta tarea se debe desarrollar **exclusivamente** con los contenidos liberados al momento de publicar el enunciado. No se permitirá utilizar contenidos que se vean posterior a la publicación de esta evaluación.
- Se encuentra estrictamente prohibido citar código que haya sido publicado **después de la liberación del enunciado**. En otras palabras, solo se permite citar contenido que ya exista previo a la publicación del enunciado.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no pase por sobre otro que sí sea especificado por enunciado.

Las tareas que no cumplan con las restricciones del enunciado obtendrán la calificación mínima (1,0).